

Name: Student ID:

ECS401: Unit 3 Lab Progress Test

This sheet is to help you and us judge your progress. You must complete it at the start of your lab session before any other activity (including getting work marked). Compare notes with others nearby and make sure everyone understands the right answers (available on the QM+ site). Talk to a demonstrator about anything that is at all unclear. Do NOT spend more than 15 minutes on answering the questions: if you don't know, leave it and get someone to explain.

1. Write a Java statement that declares a variable called `famous` and initialises it to value `true`.

Answer:

For questions 2, 3 and 4 first consider the following fragment of code and complete the dry run table in each case:

```
if (superhero == 1)           // L1
{ System.out.println("Batman"); } // L2
else
{ System.out.println("Joker"); } // L3
```

2. Complete the dry run table. What is printed to the screen if variable `superhero` holds the value 0 immediately before the above fragment is executed?

Line	superhero	superhero == 1 ?	printed to screen

Answer:

3. Complete the dry run table. What is printed to the screen if variable `superhero` holds the value 1 immediately before the above fragment is executed?

Line	superhero	superhero == 1 ?	printed to screen

Answer:

4. Complete the dry run table. What is printed to the screen if variable `superhero` holds the value 2 immediately before the above fragment is executed?

Line	superhero	superhero == 1 ?	printed to screen

Answer:

5. What boolean expression would be used to test if the String variable `actress` holds value: "Keira Knightley"?

Answer:

For questions 6, 7 and 8, consider the following fragment of code and complete the dry run table in each case:

```
if (singer > 2)                // L1
{ System.out.println("Duffy"); } // L2
else if (singer == 3)          // L3
{ System.out.println("Madonna"); } // L4
else
{ System.out.println("KT Tunstall"); } // L5
```

6. Complete the dry run table. What is printed to the screen if variable `singer` holds the value 2 immediately before the above fragment is executed?

Line	singer	singer > 2 ?	singer == 3 ?	printed to screen

Answer:

7. Complete the dry run table. What is printed to the screen if variable `singer` holds the value 3 immediately before the above fragment is executed?

Line	singer	singer > 2 ?	singer == 3 ?	printed to screen

Answer:

8. Complete the dry run table. What is printed to the screen if variable `singer` holds the value 4 immediately before the above fragment is executed?

Line	singer	singer > 2 ?	singer == 3 ?	printed to screen

Answer:

Terminology

9. **Expressions** in a program are parts of a program that evaluate to give an answer. Highlight/underline all the expressions in the following fragment of code.

```
int superhero = 10;
int supervillian = superhero - 5;
boolean saved = true;

if (superhero > supervillian)
{
    System.out.println("Saved");
    supervillian = supervillian - 5;
    saved = true;
}
else
{
    System.out.println("Oh no");
    supervillian = supervillian - 5;
    saved = ! saved;
}
```

10. A **boolean expression** is one that evaluates to a boolean (true or false) value. Highlight/underline all the boolean expressions in the following fragment of code.

```
int superhero = 10;
int supervillian = superhero - 5;
boolean saved = true;

if (superhero > supervillian)
{
    System.out.println("Saved");
    supervillian = supervillian - 5;
    saved = true;
}
else
{
    System.out.println("Oh no");
    supervillian = supervillian - 5;
    saved = ! saved;
}
```

11. An **assignment** stores a value in a variable. Its right hand side is an expression. Highlight/underline all the assignments in the following fragment of code.

```
int superhero = 10;
int supervillian = superhero - 5;
boolean saved = true;

if (superhero > supervillian)
{
    System.out.println("Saved");
    supervillian = supervillian - 5;
}
else
{
    saved = ! saved;
}
```

12. An **accessor** method is a method that either returns the value stored in a specific field of a record or stores a new value in a specific field of a record.

Write two accessor methods: one that returns the value from the `destination` field of the record below, and one that changes it to a given value, returning the new record.

```
class Train
{
    String source;
    String destination;
}
```

Answer:

